

Procedimentos de Homologação

Publicar em Tela — Opção C (Ondas A, B e C)

Branch: **Smart-Signage-Studio-Vx5** · Gerado em: 10/06/2026

1. Contexto da conversa e do projeto

Este documento consolida o trabalho realizado na branch **Smart-Signage-Studio-Vx5** para o fluxo **Publicar em Tela (Opção C)**: abas Rápido e Criar, HTML offline, assistência de IA para textos, cardápio dinâmico ao vivo e vídeo IA Premium. O núcleo *quickPublishService* foi preservado; playlist e campanha não são expostas na UI.

Commits recentes no remoto:

Commit	Conteúdo
c669af0	Ondas B e C — cardápio ao vivo (catalogRevision) e vídeo IA Premium (adapter HTTP)
68202d5	Onda A — Gerar e publicar agora (auto-publish orchestrator)
9b7e22d	Preview da mídia selecionada após upload
e344333	Fix build Linux — ResponsiveSectionNav
fbd6bdb	Anti-duplo-clique, flags ai_text_assist/ai_video, E2E modo Criar

2. O que já está implementado (código)

2.1 Onda A — Gerar e publicar agora (semi-automático)

- Serviço `autoPublishOrchestratorService`: layout → IA opcional → render HTML → `quickPublish`.
- Rota POST `/api/subscribers/:id/publish-board/:preset/auto-publish`.
- Botão **Gerar e publicar agora** + checkbox IA textos no `CreatePublishPanel`.
- Testes unitários e de integração.

2.2 Onda B — Triggers automáticos (cardápio ao vivo)

- `menuCatalogTriggerService`: `catalogRevision`, audit log, `MENU_LIVE_REFRESH_SECONDS` (5–300s, default 30).
- Triggers em `create/update/delete` de produto no `menuCatalogService`.
- API pública GET `/api/publish-board/public-menu/:subscriberId` com `meta.catalogRevision` e `refreshSeconds`.
- HTML do cardápio faz poll inteligente (refresh rápido de 2s quando revision muda).
- Aviso verde no `MenuCatalog` após salvar preço/produto.
- Sem republicação de campanha — telas HTML consultam a API periodicamente.

2.3 Onda C — Vídeo IA real (Premium)

- `aiVideoConfig` em `env.ts`: `AI_VIDEO_PROVIDER`, `AI_VIDEO_API_URL`, `AI_VIDEO_API_KEY`, `AI_VIDEO_TIMEOUT_MS`.

- httpAiVideoAdapter: POST no provedor → { videoUrl | url }.
- publishVideoAiProcessorService: checa Premium, gera, importa vídeo, cria mídia aprovada.
- Rotas queue-video-ai (201/202/403) e GET video-ai-jobs/:jobId.
- UI seleciona mediaId automaticamente quando status === completed.

3. O que ainda falta

Área	Situação	Impacto
Homologação em campo	Checklist §11 inteiro por marcar	Bloqueia release
Deploy staging	git pull + rebuild + restart com c669af0	Bloqueia todos os testes
Env vars	MENU_LIVE_*, AI_VIDEO_* não no instalador	AI não funcionam plenamente
Provedor vídeo real	Adapter genérico; falta API externa configurada	Orçário parcial na prática
Vídeo assíncrono	Geração síncrona; UI não faz poll de job	Provedores lentos quebram UX
E2E B/C	Sem mocks public-menu / queue-video-ai	Risco de regressão
Merge main	Adiado pelo usuário	Produção formal
Roadmap futuro	Framework propaganda automática, triggers, evoluções	Evoluções, não urgente

4. Ordem de importância e dependência

Execute nesta sequência. Cada passo desbloqueia o seguinte; pular etapas gera falsos negativos nos testes.

P	Passo	Depende de	Desbloqueia
P0	Deploy c669af0 no staging (pull, rebuild, restart)		Tudo
P1	Homologação manual §11 (Rápido → Criar → auto-publish)	P0	Confiança para release
P2	Configurar env (MENU_LIVE, AI_PROVIDER, AI_VIDEO_*)	P0	Testes reais B e C
P3	Cardápio ao vivo no totem físico	P1 + P2	Valor de negócio Onda B
P4	Provedor vídeo IA + teste Premium	P2 + plano Premium	Valor de negócio Onda C
P5	Fila assíncrona + poll na UI (se provedor IA)	P4	C em escala
P6	E2E automatizado Ondas B/C	P1 estável	CI sem regressão
P7	Instalador / env.example com novas vars	P2 + P4 validados	Novas instalações
P8	Merge main + release formal	P1 mín.; ideal P3+P4	Produção
P9	Roadmap futuro (triggers, framework auto)	P8	Evolução

4.1 Diagrama de dependências

P0 (Deploy) → P1 (Homologação) e P2 (Env) → P3 (Cardápio totem) e P4 (Vídeo IA) → P5 (Async, se necessário) → P6 (E2E) → P7 (Instalador) → P8 (Release) → P9 (Futuro).

5. Procedimento P0 — Deploy no staging

Pré-requisitos

- Acesso SSH ao servidor de staging (ex.: 217.216.91.135).
- Postgres com schema v6 e seed aplicados.
- Node/npm disponíveis no servidor.

Comandos (ajustar caminhos conforme instalação)

```
cd /caminho/TotemDigital
git fetch origin
git checkout Smart-Signage-Studio-Vx5
git pull origin Smart-Signage-Studio-Vx5
git log -1 --oneline
# Confirmar commit c669af0 ou posterior

cd backend && npm ci && npm run build
cd ../frontend && npm ci
# Frontend Studio:
export REACT_APP_TOTEMDIGITAL_COMPACT=true
npm run build

# Reiniciar serviços (systemd, pm2 ou script local)
sudo systemctl restart smartsignage-backend smartsignage-frontend
# ou: pm2 restart all
```

Verificação pós-deploy

- GET /health → studioMode: true.
- Login admin funciona; menu sem credenciais expostas.
- Logs: Smart Signage Studio + single_publisher.

6. Procedimento P2 — Variáveis de ambiente

Adicionar no .env do backend (e reiniciar):

```
TOTEMDIGITAL_COMPACT=true
MENU_LIVE_REFRESH_SECONDS=30

# IA textos (Onda A / suggest-copy)
AI_PROVIDER=ollama
AI_MODEL=llama3.2:3b

# Vídeo IA Premium (Onda C) – opcional até ter provedor
AI_VIDEO_PROVIDER=http
AI_VIDEO_API_URL=https://seu-provedor/video/generate
AI_VIDEO_API_KEY=sua-chave
AI_VIDEO_TIMEOUT_MS=120000
```

Ordem de configuração: primeiro MENU_LIVE_REFRESH e AI_PROVIDER; depois AI_VIDEO_* quando houver URL de provedor real.

7. Procedimento P1 — Homologação manual (Checklist §11)

Referência: docs/CHECKLIST_STUDIO_STAGING.md. Marque cada item após pass/fail documentado.

#	Teste	Passos	Critério OK	Dep.
1.1	Modo Rápido	/quick-publish?mode=quick → upload de mídia	Mídia publicada no totem sem erro	

#	Teste	Passos	Critério OK	Dep.
1.2	Modo Criar + HTML	?mode=create → preset → preview	Mídia Geral selecionada e HTML publicado	Publicação automática (não visível na UI)
1.3	Gerar e publicar agora	Botão ■ Gerar e publicar agora ■	HTML gerado e publicado em um 1:1	Que
1.4	IA textos	Sugerir textos com IA (AI_PROVIDED)	Se não configurada, isso claro se indisponível	2
1.5	Cardápio ao vivo	Publicar cardápio HTML → alterar /publish-board	Arquivo de Cardápio publicado e atualizado	4 + 302 sem republicar
1.6	Vídeo IA Premium	Plano base → mensagem Premium	Imagem selecionada e vídeo publicado	2021 de 2021 premium
1.7	Redirect	/publish-board → ?mode=create	URL redireciona corretamente	P0

Critério mínimo para merge (P8): itens 1.1, 1.2, 1.3 e 1.5 verificados; sem regressão no fluxo Rápido.

8. Procedimento P3 — Cardápio ao vivo (Onda B)

1. No modo Criar, preset Cardápio: escolher anunciante, configurar layout, Gerar animação HTML.
2. Publicar no totem de teste (contrato + totem válidos).
3. Confirmar que o player exibe o cardápio HTML (media_type: html).
4. Abrir Cardápio por cliente (/menu-catalog?subscriber=ID).
5. Editar preço de um produto e salvar — deve aparecer aviso verde de atualização ao vivo.
6. Aguardar até MENU_LIVE_REFRESH_SECONDS (default 30s) no totem.
7. Verificar novo preço na tela sem republicar campanha.
8. (Opcional) GET /api/publish-board/public-menu/:subscriberId — conferir meta.catalogRevision.

Se falhar — verificar:

- Totem acessa a mesma origem da API (CORS/rede).
- Mídia publicada é HTML com script de poll (não PNG estático).
- Cache: resposta public-menu com Cache-Control: no-cache.
- Produto isAvailable = true após edição.

9. Procedimento P4 — Vídeo IA Premium (Onda C)

1. Confirmar plano do anunciante com feature ai_video: true (seed Premium).
2. Configurar AI_VIDEO_PROVIDER=http e AI_VIDEO_API_URL no backend.
3. Provedor deve aceitar POST { brief, preset, subscriberId } e retornar { videoUrl } ou { url }.
4. No modo Criar, clicar no botão de Vídeo IA.
5. Plano base: esperar HTTP 403 e mensagem Premium.
6. Plano Premium sem provedor: HTTP 202, fila informativa.
7. Plano Premium com provedor: HTTP 201, mediaId na resposta; mídia selecionada na UI.
8. Consultar job: GET /api/subscribers/:id/publish-board/video-ai-jobs/:jobId.

P5 (se geração > timeout): implementar worker assíncrono e polling na UI usando getVideoAiJob — só após medir tempo real do provedor em P4.

10. Regras de arquitetura (preservadas)

- quickPublishService.ts — núcleo não alterado.
- Playlist/campanha não expostas na UI do Studio.
- Schema como fonte da verdade (sem migrations temporárias).
- Alterações mínimas e consistentes com convenções existentes.

11. Comandos úteis (validação local)

```
# Testes backend Ondas A/B/C
cd backend
npm test --
--testPathPattern="publish-board|publishBoardHtml|httpAiVideo|menuCatalogTrigger"

# Validar seed v6
cd database && node validate-v6.js

# E2E (com mocks)
cd e2e && npx playwright test quick-publish-create.spec.ts
```

